MODULE 10 · DOMAIN 4 - TERRAFORM CONFIGURATION

count, for_each and Dynamic Blocks

Repeating a resource, repeating a nested block, and the plan-time rule that governs both.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 10 of 27

admin@sthithapragna.com

count numbers your instances. **for_each** names them. Identity is the whole difference.

count

- **count** takes a whole number and creates that many instances of the resource.
- **count.index** gives the current instance's position, starting at zero.
- Instances form a **list**, addressed with a number in square brackets.
- Best suited to instances that are **genuinely interchangeable**, such as three identical workers.
- **Identity is positional.** Remove the first element and every later instance is renumbered.

> ON THE EXAM
>
> Use count when the instances differ only in number. The moment they have meaningful names, positions become a liability.

The conditional resource pattern

- **count = 0** creates nothing. This is how you make a resource conditional.
- The usual form pairs it with a **conditional expression**: one if a condition holds, zero otherwise.
- There is **no other way** to make a resource optional. Terraform has no if statement, as module nine established.
- The cost: the resource still uses **index zero**, so references need square bracket zero even when there is only one.
- **for_each over an empty map** achieves the same thing, and keeps named addressing.

> ON THE EXAM
>
> count = 0 is the idiomatic answer to 'create this only in production'. Expect it as a correct option.

for_each

- **for_each** accepts a **map** or a **set of strings**, and creates one instance per member.
- **It does not accept a list.** Convert one with toset, which is why that function appears so often.
- **each.key** is the map key or set member. **each.value** is the map value.
- With a **set**, key and value are identical, so use whichever reads better.
- Instances form a **map**, addressed with a string in square brackets.

> ON THE EXAM
>
> Each instance is associated with a distinct infrastructure object, tracked by key. Adding or removing a member touches only that member.

Working an example

FOR_EACH IS GIVEN	EACH.KEY AND EACH.VALUE
`toset(["a", "b"])`	key and value are both "a", then both "b"
`{ web = 2, api = 5 }`	key "web" value 2, then key "api" value 5
`["a", "b"]`	Error. A list is not accepted; wrap it in toset
`{}`	No instances. Valid, and the empty-map conditional pattern
`var.names (a list)`	Error, unless converted. This is the usual first mistake

ON THE EXAM

A set makes key and value identical. A map is the only way to carry a payload alongside the key.

The idiom you will actually write

- Real input is usually a **list of objects**, and for_each will not take a list.
- The standard move: a **for expression producing a map**, keyed by a stable field of each object.
- That gives named instances **and** the full object as each.value, so nothing is lost.
- **Choose the key carefully.** It is the instance identity, so changing it destroys and recreates.
- This is module nine's for expression and this module's for_each working together.

> ON THE EXAM
>
> Key on something the caller controls and will not change, such as a name. Never on something derived or generated.

count or for_each?

COUNT WHEN

The instances are interchangeable

- They differ only in number
- Nothing distinguishes one from another
- Conditional creation with zero
- Identity is a position

FOR_EACH WHEN

The instances are distinct

- Each has a meaningful name or key
- They may differ in configuration
- Adding or removing one is safe
- Identity is a key

THE TIEBREAKER

Ask whether removing the middle item should disturb the others. With count it does. With for_each it never does.

How instances are addressed

WRITTEN WITH	ONE INSTANCE IS
`count`	aws_instance.web[0] - a number, from zero
`for_each`	aws_instance.web["api"] - a string key
`count on a module`	module.app[0]
`for_each on a module`	module.app["eu-west-1"]
`Neither`	aws_instance.web - no index at all

ON THE EXAM

These addresses appear in plan output, in -target, in state commands and in moved blocks. Recognising them is worth marks on its own.

The plan-time rule

- **Terraform must know every instance address before it can produce a plan.** That is the whole rule.
- So **for_each keys must be known values**, and a **count number must be known**, at plan time.
- Deriving either from a **computed resource attribute** fails. Those values do not exist yet.
- The error says the value **cannot be determined until apply**, and suggests using -target.
- **The real fix is structural:** key on something you already know, such as a variable, not on a generated identifier.

> ON THE EXAM
>
> The instance key becomes part of the resource address, and addresses build the graph. Terraform cannot plan a graph it cannot name.

Two things for_each will not accept

- **Impure functions.** Keys cannot come from uuid, bcrypt or timestamp.
- Terraform defers evaluating impure functions, so their results are not available when instance addresses are decided.
- **Sensitive values.** A sensitive variable, output or resource attribute cannot be used as a for_each argument.
- That follows from addressing. A key becomes part of a resource address, and addresses are printed everywhere.
- Both restrictions are **documented rules**, not quirks. Expect them as options.

> ON THE EXAM
>
> Sensitive values cannot be instance keys, because an instance key is a name that will appear in plans, state and error messages.

Repeating a module

- **count and for_each work on module blocks too**, since Terraform 0.13.
- Everything the module creates is repeated, and addresses gain the index or key at the module level.
- Common use: **one module instance per region, per environment, or per tenant**.
- **The same plan-time rule applies.** Module keys must be known before the plan.
- Aliased providers still have to be **passed in explicitly** with the providers meta-argument, as in module two.

> ON THE EXAM
>
> Repeating a module multiplies everything inside it. That is powerful, and it makes the blast radius of a key change much larger.

Switching between them

- **count and for_each are mutually exclusive** on one resource. A block may use one, never both.
- **Changing from count to for_each changes every instance address**, from numbers to keys.
- Terraform reads that as the old instances vanishing and new ones appearing: **destroy and recreate, all of them**.
- **moved blocks** tell Terraform an instance was renamed rather than replaced. Module seventeen.
- The alternative is **terraform state mv** per instance, which is the same idea done by hand.

> ON THE EXAM
>
> The migration is safe and it is not automatic. Without moved blocks, a purely cosmetic refactor destroys production.

Dynamic blocks

- A **dynamic block** repeats a **nested block** inside a resource. It does not repeat resources.
- It takes a **for_each**, and a **content** block describing the body of each generated block.
- The label names the block being generated, and an **iterator** argument can rename the temporary variable.
- The classic use: **security group rules**, where the number of nested blocks depends on input.
- With a **set**, key and value are identical, and HashiCorp says the key **should not be used**.

> ON THE EXAM
>
> count and for_each repeat the resource. dynamic repeats a block inside it. Questions rely on that confusion.

What a dynamic block cannot generate

- **It can only generate arguments belonging to the resource type, data source, provider or provisioner.**
- **Meta-argument blocks cannot be generated.** That rules out lifecycle and provisioner blocks.
- The reason: Terraform must **process meta-arguments before it is safe to evaluate expressions**.
- So a dynamic lifecycle block is not merely discouraged. It is **impossible**, and the error says so.
- The for_each value must be a collection with **one element per desired nested block**.

> ON THE EXAM
>
> Meta-arguments are read before expressions are evaluated. Anything Terraform needs early cannot be produced by something evaluated late.

HashiCorp's own warning

- **Overuse of dynamic blocks makes configuration hard to read and maintain.** That is the documentation's phrasing.
- The recommended use: **hiding details to build a clean interface for a reusable module**.
- In a root module written for humans, a literal nested block is usually **clearer and better**.
- Dynamically generated configuration is harder to trace, which makes troubleshooting harder too.
- Same shape as the -target warning in module five. Powerful, documented, and explicitly not for routine use.

> ON THE EXAM
>
> If a scenario uses dynamic blocks to avoid typing three nearly identical rules, the answer is usually to type the three rules.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Cannot be determined until apply	A key or count derived from a computed attribute
Create this only in production	count with a conditional expression, or an empty map
Removing one item recreated the others	count where for_each was needed
A refactor proposes to destroy everything	Instance addresses changed. Use moved blocks
The number of nested rules varies by input	A dynamic block, inside one resource
A dynamic lifecycle block is rejected	Meta-arguments cannot be generated. Not a bug

ON THE EXAM

Ask what is being repeated. A resource means count or for_each. A nested block inside a resource means dynamic.

A configuration creates a set of DNS records, one per application server. An engineer writes for_each over a set built from the servers' generated private IP addresses, so each record is named after its address.

terraform plan fails, reporting that the for_each set includes values derived from resource attributes that cannot be determined until apply.

What is the correct fix?

A Apply with -target on the servers first, then plan the records

B Key the for_each on the server names, which are known, and read the IP inside the block

C Wrap the set in toset, which resolves the values at plan time

D Replace for_each with count, since count does not require known values

◆ PAUSE AND DECIDE

What is the correct fix?

A Target the servers first

The documented workaround, and a workaround. It must be repeated on every fresh apply, and module five explained why routine targeting is the wrong habit.

B Key on the known names

Correct. Instance addresses come from the names, which are known at plan time, while the unknown IP is used only inside the block body.

C Wrap in toset

toset changes the type, not the knowability. An unknown value is still unknown once it is in a set.

D Use count instead

count has the same rule. Its number must also be known at plan time, so this trades one plan-time error for another.

THE SIGNAL

Only the key must be known. Everything inside the block body may be computed, which is what makes the fix cheap.

> A module is used in development, staging and production. A bastion host and its associated firewall rules should exist in production only. The module already takes an environment variable as a string.
>
> An engineer asks how to make the bastion conditional without splitting the module in two.

What is the idiomatic approach?

A Wrap the resource in an if block that tests the environment variable

B Set count to a conditional expression giving 1 in production and 0 elsewhere

C Use a dynamic block whose for_each is empty outside production

D Declare the resource and set all its arguments to null outside production

◆ PAUSE AND DECIDE

What is the idiomatic approach?

A An if block

There is no if block in Terraform. It is declarative, and the conditional expression chooses between values rather than deciding whether code runs.

B count with a conditional

Correct, and idiomatic. count = 0 creates nothing, so the resource is present in the configuration and absent from the infrastructure.

C An empty dynamic block

Dynamic blocks repeat nested blocks inside a resource. They cannot decide whether the resource itself exists.

D All arguments null

null means 'use the provider default', so the resource is still created. It would be built with defaults, which is worse than either outcome.

THE SIGNAL

for_each over an empty map does the same job and keeps named addressing. Both are correct; count is the one questions expect.

> A module author wants callers to be able to enable prevent_destroy on the resources the module creates. She writes a dynamic block with the label lifecycle, whose for_each is empty when the feature is off.
>
> Terraform rejects the configuration.

Why, and what should she do instead?

A Dynamic blocks cannot generate meta-argument blocks. lifecycle must be written literally

B The label must be quoted, and lifecycle is a reserved word requiring an iterator

C Dynamic blocks require a non-empty for_each, so an empty collection is invalid

D prevent_destroy is not supported inside modules, only in root configurations

◆ PAUSE AND DECIDE

Why, and what should she do instead?

A Meta-arguments cannot be generated

Correct. Terraform processes meta-arguments before it is safe to evaluate expressions, so lifecycle and provisioner blocks cannot be produced dynamically.

B Quoting and iterator

Invents a rule. Labels are quoted anyway, and no keyword becomes valid by adding an iterator argument.

C Empty for_each invalid

An empty collection is perfectly valid and simply generates no blocks. That is a normal and useful pattern.

D Not supported in modules

prevent_destroy works anywhere a lifecycle block is valid. Module scope has nothing to do with it.

THE SIGNAL

prevent_destroy also requires a literal value, so it cannot be driven by a variable either. Module eleven returns to this.

> A team has nine production databases created with count over a list of names. After the incident described in module nine, they decide to switch to for_each over a set of the same names.
>
> The change is purely a refactor. The plan proposes to destroy all nine databases and create nine new ones.

What should they do before applying?

A. Nothing. The plan is wrong and Terraform will reconcile the addresses on apply

B. Add moved blocks mapping each old index to its new key, then re-plan

C. Apply with -refresh=false so the existing instances are not re-read

D. Revert to count, since migrating between the two is not supported

◆ PAUSE AND DECIDE

What should they do before applying?

A **The plan is wrong**

The plan is correct. Every instance address changed, so from Terraform's view nine objects are no longer declared and nine new ones are. It will do exactly what it says.

B **Add moved blocks**

Correct. A moved block tells Terraform an instance was renamed rather than replaced, so state is updated and no infrastructure is touched.

C **Apply with -refresh=false**

Refresh controls re-reading remote objects. It has no effect on addressing, and would only make the plan less accurate.

D **Migration unsupported**

It is supported and routine. moved blocks exist for precisely this refactor, and terraform state mv did the job before them.

THE SIGNAL

Terraform compares addresses, not intentions. A refactor that changes addresses is indistinguishable from a deletion unless you say otherwise.

Module 10 recap

- **count numbers instances, for_each names them.** Positions shift, keys do not.
- **for_each takes a map or a set of strings.** Never a list, hence toset.
- **Keys and counts must be known at plan time,** because they become addresses in the graph.
- **No impure functions and no sensitive values as for_each keys.** Both are documented rules.
- **count = 0 is how you make a resource conditional.** There is no if statement.
- **dynamic repeats a nested block, not a resource,** and cannot generate meta-argument blocks.

> ON THE EXAM
>
> Switching between count and for_each rewrites every address. Use moved blocks, or Terraform will destroy and recreate everything.